JAVA

Tp 1 : Les fondamentaux : de la logique impérative aux objets

---

Programmation impérative • POO • JDK / JRE / JVM • Premier TP (Scanner)

Support de cours — Document rédigé en noir et blanc

Equipe : Créateur Mr Moez Ben Rkaya & Nadia Boulifa

1. Deux façons de penser un programme

Programmation impérative vs Programmation orientée objet

① Impérative	② Orientée Objet
Une suite d'instructions qui modifient l'état des variables, étape par étape. — Le programme = une recette de cuisine — Fonctions qui agissent sur des données — Question clé : « Que dois-je faire ? »	Le programme est organisé autour d'objets qui regroupent données et comportements. — Le programme = une équipe qui coopère — Objets = données + méthodes — Question clé : « Quels objets, et que savent-ils faire ? »

```
int l = 5, w = 3;
int surface = l * w;
System.out.println(surface);
```

```
class Rectangle {
  int l, w;
  int surface(){ return l*w; }
}
```

2. Compilation vs Interprétation

Comment le code source devient-il exécutable ?

Programmation impérative	Programmation orientée objet (Java)
Toujours compilée : le compilateur traduit directement le code source en code machine (exécutable natif). code source → compilateur → exécutable (.exe)	Interprétée : javac compile d'abord vers un bytecode, puis la JVM interprète (exécute) ce bytecode. .java → javac → .class → JVM (interprète)

À retenir

Tout langage interprété est d'abord compilé (vers un bytecode ou un code intermédiaire) — mais l'inverse est faux : un langage compilé n'est pas forcément interprété ensuite.

3. La classe et l'objet

Deux notions centrales de la POO

Classe

Le plan, le modèle. Elle décrit les attributs et les méthodes que ses futurs objets posséderont.

Objet

Une instance concrète construite à partir de la classe : elle existe en mémoire avec ses propres valeurs.

Analogie

La classe « Voiture » est un moule à gâteau. Chaque objet (voiture1, voiture2…) est un gâteau différent sorti du même moule : même moule, valeurs différentes (couleur, vitesse…).

```
class Voiture {
  String couleur;
  int vitesse;
  void accelerer(){ vitesse += 10; }
}

Voiture v1 = new Voiture();
v1.couleur = "rouge";
```

4. JDK, JRE, JVM : qui fait quoi ?

Trois couches imbriquées, du développement à l'exécution

JVM	Java Virtual Machine — Exécute le bytecode (.class). Rend Java portable : « write once, run anywhere ».
JRE	Java Runtime Environment — JVM + bibliothèques standard. Suffit pour EXÉCUTER un programme déjà compilé.
JDK	Java Development Kit — JRE + outils (javac, débogueur...). Nécessaire pour DÉVELOPPER.

5. La variable d'environnement PATH

Pourquoi « java » et « javac » doivent être reconnus dans le terminal

Le PATH est la liste des dossiers où le système cherche les programmes à exécuter. Si le dossier « bin » du JDK n'y figure pas, le terminal répond : « 'java' n'est pas reconnu en tant que commande interne... »

Windows (cmd)	**Linux / macOS (terminal)**
1. Panneau de configuration → Système → Paramètres système avancés → Variables d'environnement. 2. Dans « Variables système », sélectionner Path → Modifier → Nouveau. 3. Ajouter le chemin du dossier bin du JDK : C:\Program Files\Java\jdk-21\bin Vérifier : java -version	1. Ouvrir le fichier de configuration du shell (~/.bashrc ou ~/.zshrc). 2. Ajouter la ligne export ci-dessous à la fin du fichier. 3. Recharger la configuration pour appliquer les changements : export PATH=$PATH:/usr/lib/jvm/jdk-21/bin source ~/.bashrc

6. TP n°1 — Compiler et exécuter en ligne de commande

De l'écriture du code au résultat affiché dans le terminal

1	**Vérifier l'installation** java -version / javac -version
2	**Écrire le programme** fichier Bonjour.java
3	**Compiler** javac Bonjour.java → génère Bonjour.class
4	**Exécuter** java Bonjour

7. La syntaxe de départ : public static void main

Décortiquons mot par mot la ligne la plus recopiée du Java

```
public static void main(String[] args) {
```

public	Modificateur d'accès : la méthode est visible de partout, y compris par la JVM qui doit pouvoir l'appeler depuis l'extérieur de la classe.
static	La méthode appartient à la classe elle-même, pas à un objet précis : la JVM peut donc l'exécuter sans créer d'instance.
void	Cette méthode ne renvoie aucune valeur au programme appelant (pas de return utile).
main	Nom réservé et attendu par la JVM : c'est le tout premier point d'entrée exécuté au lancement du programme.
String[] args	Paramètre : un tableau de chaînes de caractères contenant les arguments passés au programme depuis la ligne de commande.

À retenir

Cette signature ne change (presque) jamais dans un programme Java simple.

8. Quatre exercices : utilisation avancée de Scanner

Du plus simple au plus complexe — à réaliser en ligne de commande

[FACILE] Exercice 1 — Salutation personnalisée

Demander le prénom (nextLine) puis l'âge (nextInt) de l'utilisateur. Afficher : « Bonjour X, dans 10 ans vous aurez Y ans. »

[FACILE] Exercice 2 — Moyenne de 3 notes

Lire trois notes avec nextDouble(), calculer leur moyenne et afficher « Admis » si elle est ≥ 10, sinon « Ajourné ».

[INTERMÉDIAIRE] Exercice 3 — Le piège nextInt() + nextLine()

Lire un entier avec nextInt() puis une phrase avec nextLine() : le nextLine() semble « sauté ». Expliquer pourquoi (buffer) et corriger avec un nextLine() supplémentaire.

[DIFFICILE] Exercice 4 — Saisie contrôlée par sentinelle

Lire des entiers en boucle jusqu'à ce que l'utilisateur tape 0. Utiliser hasNextInt() pour éviter les erreurs de saisie (InputMismatchException), puis afficher la somme et la moyenne.

9. À vous de jouer

Exercice : demandez deux nombres au clavier avec Scanner, puis affichez leur somme.

1	**Utilisez nextInt() pour lire chaque nombre**
2	**N'oubliez pas de compiler avec javac avant d'exécuter avec java**
3	**Testez avec des valeurs différentes pour vérifier le résultat**